

Matemáticas y música: sintetizando música

María de Lourdes Oros Barrón.

Noviembre del 2016.

1 Introducción

Nuestro interés reside en construir un modelo para una señal dada $s(t)$ que viene de un instrumento musical. Naturalmente, para extraer más información de la señal, pensamos en llevarla a otra que dependa de las frecuencias de la misma, esto nos lleva a considerar la transformada de Fourier de $s(t)$. Por otro lado, para representar los patrones de la señal, nos apoyamos de las **hidden variables**:

- El número m de notas.
- Los tiempos $t_i = k_i \Delta t$ en que las notas empiezan, $1 < k_1 < k_2 < \dots < k_r < N$.
- Las frecuencias ω_i de las notas en Hertz, o bien los periodos $p_i = \frac{1}{\Delta t \cdot \omega_i} \in \mathbb{Z}$.

Se supondrá en adelante que la señal de sonido ha sido muestreada de manera discreta.

Eventualmente definimos una función de densidad de probabilidad $p(\vec{s}, m, \vec{t}, \vec{p})$ sobre todas las variables para con ello reconstruir una nota de la señal observada $\vec{s}_{obs}$.

En este trabajo se implementa una de las maneras de crear un sintetizador de música, en principio creando sonido a partir de una muestra que sigue el modelo propuesto. Se sigue fielmente la idea del autor David Mumford en su libro Pattern Theory descrita en el capítulo 2.

Introducimos antes herramientas que usaremos adelante.

2 Distribuciones Gaussianas

Sea $\vec{x} \in \mathbb{R}^n$. Definimos la distribución Gaussiana en $\mathbb{R}^n$ por su función de densidad

$$p(\vec{x}) = \frac{1}{Z} e^{-(\vec{x} - \vec{m})^t Q (\vec{x} - \vec{m})/2}$$

donde $\vec{m} \in \mathbb{R}^n$, Q es una matriz cuadrada de $n \times n$, definida positiva, simétrica y Z es una constante tal que $\int p(\vec{x}) d\vec{x} = 1$.

La distribución Gaussiana tiene las siguientes propiedades:

- $Z = (2\pi)^{\frac{n}{2}} \det(Q)^{-1}$.
- $\int (\vec{x} - \vec{m}) p(\vec{x}) d\vec{x} = 0$ donde $\vec{m}$ es la media.
- Si $C_{ij} = \int (\vec{x}_i - \vec{m}_i)(\vec{x}_j - \vec{m}_j) p(\vec{x}) d\vec{x}$ es la matriz de covarianza entonces $C = Q^{-1}$.

3 La transformada de Fourier.

Si $(f_0, f_1, \dots, f_N)$ es una sucesión de funciones entonces su transformada discreta de Fourier así como su inversa son respectivamente:

$$\widehat{f_m} = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \exp\left(-2\pi i \frac{nm}{N}\right) f_n;$$
$$f_m = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} \exp\left(2\pi i \frac{nm}{N}\right) \widehat{f_n}.$$

4 Modelos Gaussianos para notas musicales

4.1 Modelos Gaussianos para señales cíclicas finitas.

Sea $\vec{s} = (s_0, \dots, s_{N-1})$ una señal periódica y $\vec{f}^k = \frac{1}{\sqrt{N}}(1, e^{2i\pi \frac{k}{N}}, \dots, e^{2i\pi \frac{k(N-1)}{N}})$ para $0 \leq k \leq N-1$ la base de Fourier.

Podemos entonces expresar la señal dada en esta base, usando la transformada inversa, como:

$$\vec{s} = \sum_{l=0}^{N-1} \vec{s}_l \vec{f}^l$$

Teorema Una distribución Gaussiana p_Q es estacionaria si y solo si $\vec{m}$ es constante y Q es una matriz bandada hermitiana (Q_{ij} depende solo de $j \bmod N$ y $a_j = \overline{a_{N-j}}$).

4.2 El caso de una nota musical.

Se construye un modelo Gaussiano de una nota.

Si ω es la frecuencia fundamental de la nota, entonces $p = 1/\omega$ es su periodo. Si $s(t)$ es la señal entonces $s(t+p) \approx s(t)$. Consideramos entonces que la señal es periódica y expandimos en series de Fourier con frecuencias n/p donde n es el tamaño del muestreo.

Formalizamos la propiedad $s(t+p) \approx s(t)$ suponiendo que el valor esperado $\int (s(t+p) - s(t))^2 dt$.

Tomemos un muestreo aleatorio de la señal s y supongamos que $s_{N+k} = s_k$ para algún N y sea $q = N/p$ el número de ciclos, para algún natural p .

La función de densidad de una distribución Gaussiana para s es:

$$p_{a,b}(s) = \frac{1}{Z} e^{-a \sum_{k=0}^{N-1} (s(k) - s(k+p))^2 / 2 - b \sum_{k=0}^{N-1} s(k)^2 / 2} = \frac{1}{Z} e^{-\vec{s}^t Q \vec{s} / 2}$$

donde $a \gg b > 0$, $Q_{ii} = 2a + b$, $Q_{i,i+p} = -a$. $p_{a,b}$ es una distribución estacionaria.

En las bases de Fourier

$$p_{a,b}(s) = \frac{1}{Z} \prod_l e^{-(b+4a \sin(\frac{\pi p l}{N}))^2 |\hat{s}(l)|^2 / 2}$$

4.3 Discontinuidades en señales de dimensión uno

Las notas se descomponen en compases, tonos, melodías etc. Naturalmente las señales tienen estructuras jerárquicas separadas por varios tipos de discontinuidades a las que llamaremos patrones geométricos. Tales discontinuidades son consideradas en este contexto como saltos muy grandes en los datos, esto se estudia vía kurtosis.

5 El modelo para música

Sea $s(t)$ una señal, y como antes, supongamos que la hemos discretizado. Supongamos ahora que m el número de notas, $\vec{t}$ el vector de tiempos en que inician las mismas, $\vec{p}$ el vector de periodos están fijos. De la ley de probabilidad condicional

$$p(\vec{s}, m, \vec{t}, \vec{p}) = p(\vec{s} | m, \vec{t}, \vec{p}) p(m, \vec{t}, \vec{p}).$$

De esta manera, la distribución de probabilidad $p(\vec{s}, m, \vec{t}, \vec{p})$ se puede descomponer como sigue:

$$p(\vec{s}, m, \vec{t}, \vec{p}) = \prod_{l=1}^m p((\vec{s} | I_l) | p_l, t_l, t_{l+1}) p(\vec{p}, \vec{t}, m)$$

donde $I_l = \{t | t_l \leq t < t_{l+1}\}$ y t_l con $l \in \{1, \dots, m\}$ son los intervalos de tiempo en que inician las notas. En la expresión anterior se supone que en cada periodo $[t_l, t_{l+1})$ las notas son independientes e idénticamente distribuidas con distribución Gaussiana.

Se construyó un modelo Gaussiano para las notas de la forma

$$p((\vec{s} | I_l) | p_l, t_l, t_{l+1}) = \frac{1}{Z} \exp(-a \sum_{n=t_l}^{t_{l+1}-p_l-1} (s_{n+p_l} - s_n)^2 / 2 - b \sum_{n=t_l}^{t_{l+1}-1} s_n^2 / 2)$$

con $a \gg b$. Un modelo geométrico muy simple para esto se obtiene tomando $\vec{t}$ con distribución Poisson y cada p_l independiente de los otros periodos.

Ahora que podemos muestrear de este modelo mostraremos un ejemplo creando un sintetizador de música. Se describe en el apartado siguiente.

6 Sintetizando música

De acuerdo a David Mumford, en su libro Pattern Theory, una manera de sintetizar es tomando una muestra aleatoria de un proceso Gaussiano estacionario, eligiendo coeficientes de Fourier de una distribución Gaussiana con media cero y cualquier varianza: los coeficientes de Fourier son independientes. Sintetizaremos ruido de color como sigue: Tomamos la parte real y la parte imaginaria de los coeficientes de Fourier $\hat{s}_k, 0 \leq k < N$ variables aleatorias con media 0 y desviación estándar $\frac{1}{\min(k, N-k)^\lambda}$ y $\hat{s}_0 = 0$.

Hacemos esto para $N = 1024$ y $\lambda \in \{0, 0.5, 1, 2\}$ y graficamos los resultados.

Código: escrito en Jupyter.

```
# Importamos librerias:
import numpy as np
#import sounddevice as sd
import matplotlib.pyplot as plt
from IPython.display import Audio
%matplotlib inline

# variables iniciales:
N = 1024 # samples
lmbd = [0, 0.5, 1, 2] # Lambda: power law
n = len(lmbd)
x = np.linspace(0, N, N, endpoint=False) # x-axis

# Funcion ruido de color:
def colored_noise(N, lmbd):
    n = len(lmbd)
    y = np.zeros((n, N)) # y-axis

    # y = f(x) axis:
    for i in range(n):
        for k in range(1, N):
            y[i, k] = np.random.normal(0., 1. / (np.min((k, N - k))) ** lmbd[i])

    # Transformada rapida de Fourier:
    f_vec = np.array([ np.fft.fft(y[i]) for i in range(n) ])

    return f_vec

# Ruido de color!
f_vec = colored_noise(N, lmbd)
```

```
#PLOT
# Parte real:
fig, ax = plt.subplots(figsize=(20,15))
for i in range(n):
    plt.subplot(2, 2, i+1)
    plt.title("Real part of FFT for lambda = {}".format(lmbd[i]))
    plt.plot(x, f_vec[i].real, "lightseagreen")
    plt.xlim(0, N)
```

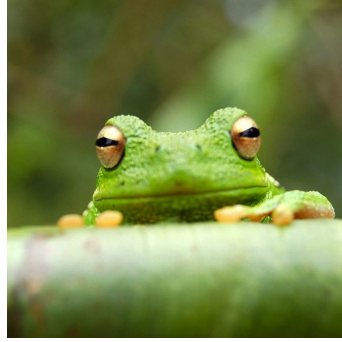


Figure 1: This frog was uploaded via the project menu.

```
#PLOT
# Parte imaginaria:
fig, ax = plt.subplots(figsize=(20,15))
for i in range(n):
    plt.subplot(2, 2, i+1)
    plt.title("Imaginary part of FFT for lambda = {}".format(lmbd[i]))
    plt.plot(x, f_vec[i].imag, "lightseagreen")
    plt.xlim(0, N)
```

Observaciones.

Las frecuencias más bajas vienen de los coeficientes de Fourier con índices cercanos a cero y N .
 Sintetizando notas para música Sintetizando notas para música tomando una distribución Gaussiana.
 $s(k)$ se asume de periodo N , i.e., $s(N+k) = s(k)$. Tomamos ahora $N = 8192$ y $p = 32$.

```
def concatenate_the_noise(period, N, p, index, real=True):
    if real:
        f_vec = np.concatenate((period[index].real, period[index].real))
    else:
        f_vec = np.concatenate((period[index].imag, period[index].imag))

    # Concatenamos muchas veces:
    power = int(np.ceil(np.log2(p)))
    for i in range(power):
        f_vec = np.concatenate((f_vec, f_vec))

    return f_vec[:int(N*p)]

def listen_the_synth(sampling):
    wave = 10000 * sampling # factor a escala perceptible humana
    wav_wave = np.array(wave, dtype=np.int16)
    #sd.play(wav_wave, blocking=True)
    return wav_wave

# variables:
N = 8192 # longitud del muestreo
p = 32 # Submuestreo, particionando en periodos
q = N/p # cicclos (256)

# sucesoion de numeros equiespaciados
x = np.linspace(0, N, N, endpoint=False)
period = colored_noise(int(q), lmbd)

# Concatenando para obtener la sintesis
```

```
# Index es para diferentes lambdas
synth = concatenate_the_noise(period, p, q, 2, real=True)

# Plot y sonido
fig, ax = plt.subplots(figsize=(15,5))
plt.plot(x, synth, "lightseagreen", linewidth=2)
plt.xlim(0, 3*q)

sound = listen_the_synth(synth)
Audio(sound, rate=44100)
```

```
# Ejemplo, onda sinusoidal.
#variables:
#sd.default.samplerate = 44100
time = 2.0
frequency = 440

# Genera tiempo de muestreo entre 0 y 2 segunds
samples = np.arange(44100 * time) / 44100.0
# onda  $w(t) = A \sin(2\pi f t)$ 
wave = 10000 * np.sin(2 * np.pi * frequency * samples)
# formato wav (16 bits)
wav_wave = np.array(wave, dtype=np.int16)

#sd.play(wav_wave, blocking=True)
Audio(wav_wave, rate=44100)

Fs = 44100.0; # rango de muestreo
Ts = 1.0/Fs; # sampling intervalo
t = np.arange(0,time,Ts) # time vector

ff = 5; # frecuencia de la senal
y = np.sin(2 * np.pi * frequency * samples)

n = len(y) # longitud de la senal
k = np.arange(n)
T = n/Fs
frq = k/T # ds lados de la frecuencia
frq = frq[range(int(n/2))] # un lado de la frecuencia

Y = np.fft.fft(y)/n # fft
Y = Y[range(int(n/2))]

fig, ax = plt.subplots(3, 1, figsize=(15,5))
ax[0].plot(t, y, 'steelblue')
ax[0].set_xlabel('Time')
ax[0].set_ylabel('Amplitude')
ax[0].set_xlim(0, time/64)
ax[1].plot(frq,Y.real,'magenta') # grafica del espectro
ax[1].set_xlabel('Freq (Hz)')
ax[1].set_ylabel('Y(freq)')
ax[1].set_xlim(300, 900)
ax[2].plot(frq, abs(Y), 'magenta') # grafica del espectro
ax[2].set_xlabel('Freq (Hz)')
ax[2].set_ylabel('|Y(freq)|')
ax[2].set_xlim(300, 900)
```

Dada una señal, esta se clasifica como ruido cuando la gráfica de la misma carece de "forma," en el sentido de que no muestra patrones repetitivos (en la gráfica) a simple vista, el ruido no es agradable

a nuestros oídos.

Las señales clasificadas como "sonido" adoptan una forma más bonita en el sentido de que su gráfica tiene estructura periódica y suave.

References

- [1] Pattern Theory. David Mumford. *Ch2*